

CM 5 - LES LIBRAIRIES NUMPY ET SCIPY

PROGRAMMATION PYTHON – MASTER 1 MAS

Romain Tavenard

2026

✉ romain.tavenard@univ-rennes2.fr



Calcul numérique en Python :

- **numpy** : calculs numériques de base (opérations arithmétiques, ...)
- **scipy** : fonctions avancées (calculs de distance, Transformée de Fourier, résolution de systèmes d'équations, etc.)

Calcul numérique en Python :

- **numpy** : calculs numériques de base (opérations arithmétiques, ...)
- **scipy** : fonctions avancées (calculs de distance, Transformée de Fourier, résolution de systèmes d'équations, etc.)

Ces opérations déjà implémentées sont **beuuuuuuuuucoup** plus rapides que si vous les codiez vous-même.

But du jeu : se reposer sur les opérateurs existants tant que possible
⇒ **NE PAS** coder des boucles sur les indices de tableaux **numpy**

Le type de base des objets manipulés est `numpy.ndarray` (tableau à n dimensions) :

- 1 dimension : vecteur
- 2 dimensions : matrice
- 3 dimensions et plus : tenseur

Le terme dimension ici n'a pas le même sens qu'en maths !

```
print(x.ndim)
```

Pour obtenir la taille d'un `numpy.ndarray` :

```
print(x.shape) # Affiche un tuple
```

Soit la matrice suivante :

```
import numpy as np
m = np.array([[1, 2, 3],
              [0, 0, 0],
              [9, 8, 7]])
u = np.array([[6, 2, 3],
              [0, 4, 1],
              [0, 8, 7]])
```

“

Que vaut $m + 1$?

Que vaut $m + u$?

*Que vaut $m * 1$?*

*Que vaut $m * u$?*

Que vaut $m @ u$?

”

Soit la matrice suivante :

```
import numpy as np  
m = np.array([[1, 2, 3],  
              [9, 8, 7]])
```

“ Que vaut `m.sum()` ?
Que vaut `m.sum(axis=0)` ?
Que vaut `m.sum(axis=1)` ?

”

On peut utiliser les mêmes stratégies d'indilage que pour les listes :

```
import numpy as np
m = np.array([[1, 2, 3],
              [9, 8, 7]])

print(m[0, :])      # Équivalent à print(m[0])
print(m[:, 0])
print(m[1:, :-1])
```

Petite spécificité numpy :

```
import numpy as np
v = np.array([1, 2, 3])
w = np.array([0, 10])

print(v.shape)           # (3, )
print(v[:, None].shape)  # (3, 1)
print(v[None, :].shape)  # (1, 3)
```

Indiçage avec **None** : pour quoi faire ???

```
mat = np.zeros((3, 2))
for i in range(len(v)):
    for j in range(len(w)):
        mat[i, j] = v[i] + w[j]

# Remplacé par :
mat = v[:, None] + w[None, :]
#      ^ (3, 1)      ^ (1, 2)
```

$$\begin{pmatrix} v_0 + w_0 & v_0 + w_1 \\ v_1 + w_0 & v_1 + w_1 \\ v_2 + w_0 & v_2 + w_1 \end{pmatrix}$$

Cette astuce fonctionne pour n'importe quelle dimension.

“

1. Si m est de shape $(3, 2)$ et que n est de shape $(3, 4)$, à quoi ressemblera $m[:, \text{None}, :] + n[:, :, \text{None}]$?
2. Écrivez une fonction qui prend en entrée deux vecteurs numpy et retourne la matrice des distances Euclidiennes paires à paires de leurs éléments :

$$\begin{pmatrix} d(v_0, w_0) & d(v_0, w_1) \\ d(v_1, w_0) & d(v_1, w_1) \\ d(v_2, w_0) & d(v_2, w_1) \end{pmatrix}$$

”